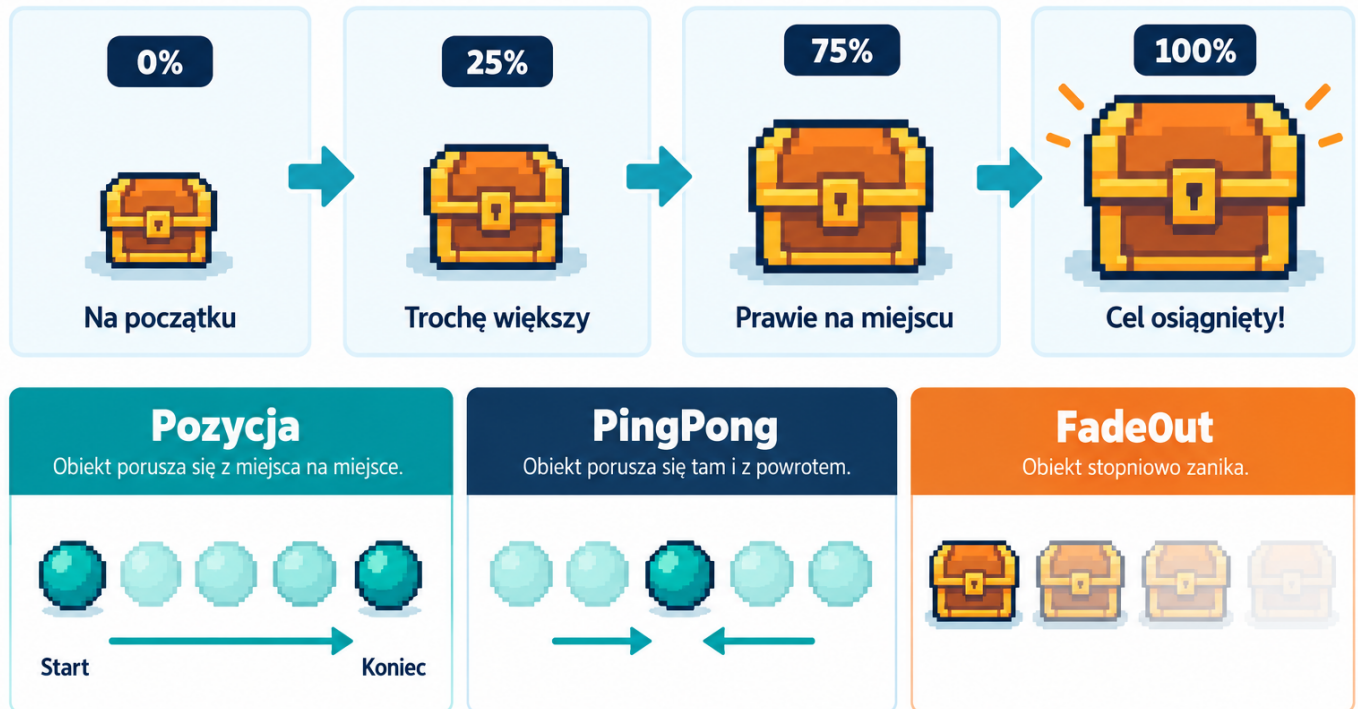


Animacja właściwości w czasie

Tween: od początku do celu

Płynna animacja zmienia właściwości obiektu w czasie.

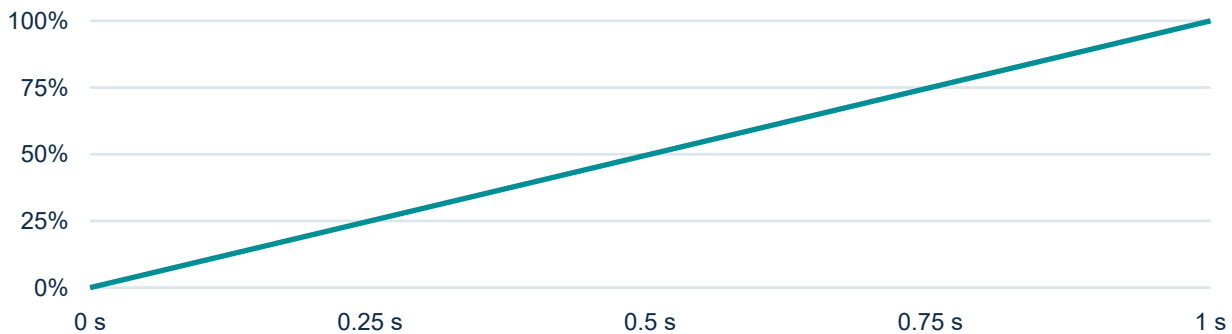


Tween zmienia wartość od początku do celu przez określony czas. Może poruszać obiekt, zmieniać skalę albo obrót. **Postęp czasu** to elapsed / duration, zwykle od 0 do 1. Easing zamienia ten postęp na przebieg zmiany wartości.

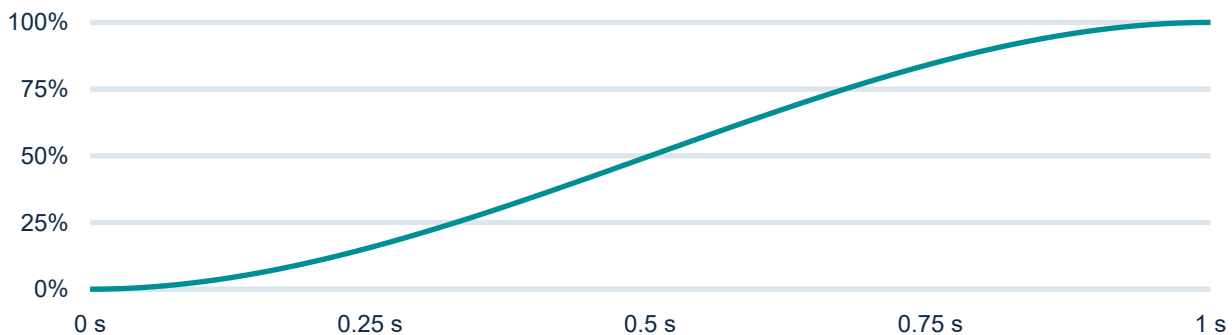
GameLab ma gotowe position, scale, rotation i shake. PingPong i FadeOut w tym kursie to **nasze komponenty** oparte na dostępnych mechanizmach - nie nazwy gotowych metod silnika.

Linear i smooth: czytaj wykres

Linear: wartość rośnie równomiernie



Smooth: wolny start i wolne zakończenie



Oś X: czas. Oś Y: część zmiany wartości, od 0% do 100%. Nachylenie wykresu pokazuje szybkość zmiany, a nie pozycję na planszy. Przy smooth w połowie czasu osiągasz 50% zmiany; na początku i końcu ruch jest wolniejszy.

Funkcje silnika: linear = t ; smooth = $t^2 \times (3 - 2t)$. Dla $t = 0,25$: linear daje 25%, a smooth 15,625%. Dla ruchu x od 100 do 300 są to odpowiednio $x = 150$ i $x = 131,25$.

Pozycja, skala, obrót, drżenie

Fragmenty uruchamiaj po stworzeniu obiektu, np. w `onCreate()`. Cel pozycji podajesz w pikselach świata, czas w sekundach. Tween sam aktualizuje obiekt w kolejnych klatkach.

```
const tween = GameLab.Tweens.position(
  object, 300, 100, 1
);
tween.easing = 'linear';

const pulse = GameLab.Tweens.scale(
  object, 1.3, 1.3, 0.3
);
pulse.easing = 'smooth';

GameLab.Tweens.rotation(object, Math.PI / 2, 0.5);
GameLab.Tweens.shake(object, 6, 0.2);
```

Skala 1 oznacza rozmiar bazowy, 1,3 to 130%. Obrót jest w radianach: $\text{Math.PI} / 2$ to 90°. Shake zmienia `visualOffset`, nie fizyczną pozycję. Domyślny easing silnika to `smooth`; ustawiaj go jawnie w ćwiczeniach.

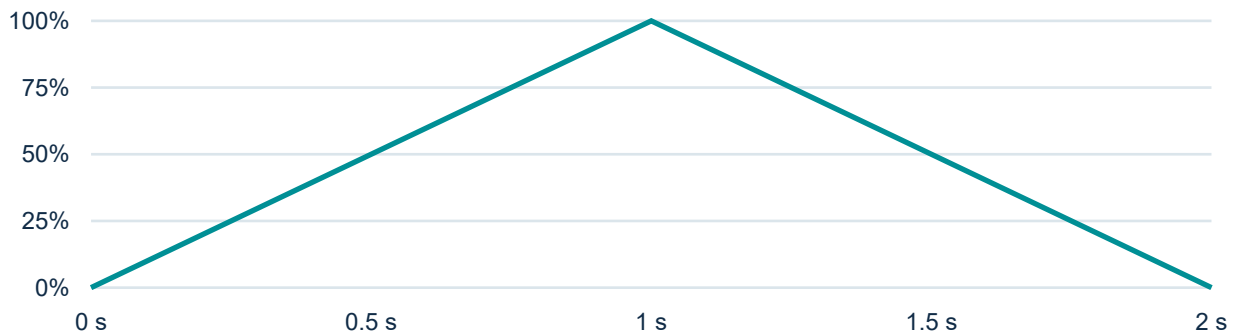
Koniec i przerwanie

```
// Uchwył przechowaj np. jako this.tween.
if (this.tween.completed) {
  // Animacja doszła do celu.
}
// Gdy chcesz przerwać:
this.tween.cancel();
```

Nowy tween tej samej właściwości obiektu zastępuje poprzedni. Nie twórz go bezwarunkowo w `onUpdate`: ciągle zaczynałby od nowa. Nie steruj jednocześnie pozycją przez tween i kontroler ruchu. Animacja skali nie zmienia rozmiaru collidera.

Tam i z powrotem

Pozycja względem drogi A → B → A



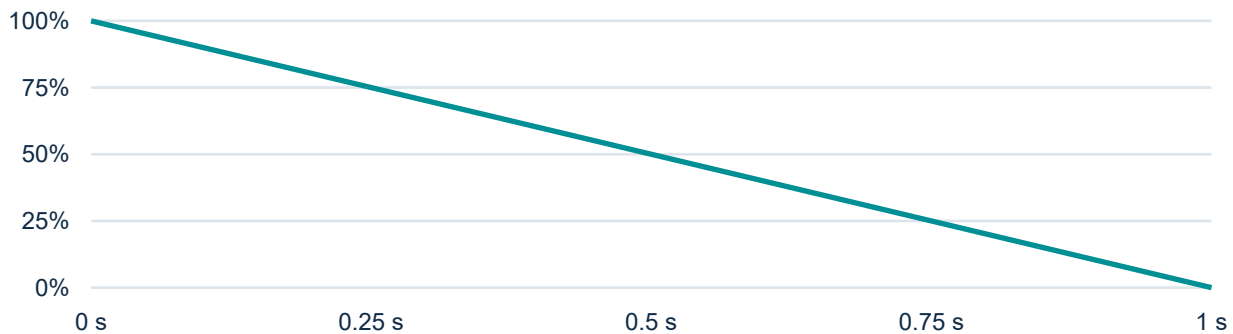
To wykres **pozycji na odcinku**, nie całkowitego postępu czasu. Po 1 s obiekt jest w B, po 2 s wraca do A. Postęp pełnego cyklu nadal rośnie od 0% do 100%. Wariant linear zawraca gwałtownie; smooth spowalnia przy końcach.

```
class PingPong extends GameLab.Component {
  onCreate() {
    this.a = this.transform.x;
    this.b = this.a + 200;
    this.toB = true;
    this.startLeg();
  }
  startLeg() {
    const target = this.toB ? this.b : this.a;
    this.tween = GameLab.Tweens.position(
      this.gameObject, target, this.transform.y, 1
    );
    this.tween.easing = 'linear';
  }
  onUpdate() {
    if (!this.tween.completed) return;
    this.toB = !this.toB;
    this.startLeg();
  }
}
```

Dodaj komponent do obiektu z rendererem: `object.addComponent(new PingPong())`. Każde ramię trwa 1 s. Zmień easing na smooth i porównaj zawracanie.

Znikanie to zmiana przezroczystości

Opacity maleje: 100% → 0%



Postęp czasu rośnie, lecz opacity maleje. W obecnym GameLab nie ma `Tweens.fadeOut` ani `Sprite.opacity`. Poniższy własny komponent wygasza **kolor ShapeRenderer** przez zapis `rgba`. Nie stosuj go do tekstury `Sprite`.

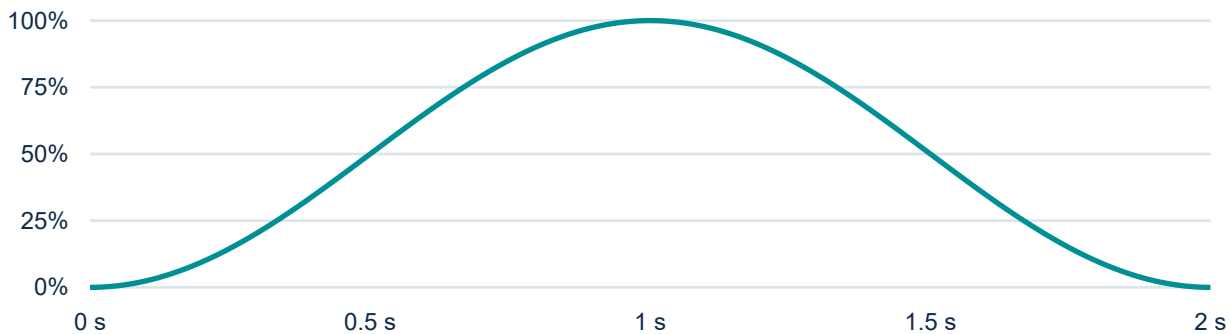
```
class FadeOut extends GameLab.Component {
  onCreate() {
    this.elapsed = 0;
    this.duration = 1;
    this.renderer = this.gameObject.getComponent(
      GameLab.ShapeRenderer
    );
    if (!this.renderer) {
      throw new Error('FadeOut wymaga ShapeRenderer');
    }
  }
  onUpdate(delta) {
    this.elapsed = Math.min(
      this.duration, this.elapsed + delta
    );
    const t = this.elapsed / this.duration;
    const opacity = 1 - t;
    this.renderer.color =
      `rgba(255, 140, 40, ${opacity})`;
    if (t === 1) this.enabled = false;
  }
}
```

Dodaj do pomarańczowego kształtu: `object.addComponent(new FadeOut())`. Niewidoczny obiekt nadal istnieje. Jeśli to efekt jednorazowy, możesz na końcu wywołać `this.gameObject.destroy()` zamiast tylko wyłączyć komponent.

FadeIn, pulsowanie i kołysanie

FadeIn: użyj $\text{opacity} = t$ zamiast $1 - t$. Ustaw color na $\text{rgba}(\dots, 0)$ już w `onCreate`, aby pierwszy obraz był przezroczysty. **FadeOut smooth:** oblicz $e = t \times t \times (3 - 2 \times t)$, potem $\text{opacity} = 1 - e$.

Pulsowanie sinusoidalne: minimum → maksimum → minimum



```
class Pulse extends GameLab.Component {
  onCreate() { this.elapsed = 0; }
  onUpdate(delta) {
    this.elapsed += delta;
    const cycle = this.elapsed / 2;
    const wave = (1 - Math.cos(
      cycle * Math.PI * 2
    )) / 2;
    const scale = 1 + 0.2 * wave;
    this.transform.scale.x = scale;
    this.transform.scale.y = scale;
  }
}
```

To animacja cykliczna napisana ręcznie, nie kolejna gotowa metoda Tweens. Skala zmienia się od 1 do 1,2, a pełny cykl trwa 2 s. Kołysanie obrotu można uzyskać przez $\text{rotation} = \text{Math.sin}(\text{czas} \times \text{częstotliwość}) \times \text{amplituda}$; amplitudę podaj w radianach.

Zaplanuj animację, potem ją napisz

1. Przesuń kształt z $x = 100$ do $x = 300$ w 2 s. Porównaj linear i smooth.
2. Dodaj PingPong do skrzynki. Jeden odcinek ma trwać 0,8 s.
3. Dodaj FadeOut do ShapeRenderer i wydłuż czas do 2 s.
4. Zmień FadeOut w FadeIn.
5. Dodaj Pulse do monety. Zmień maksymalną skalę na 1,4.

Mini-projekt: reakcja skrzynki

W onCreate przygotuj widoczną skrzynkę. Po nowym naciśnięciu spacji uruchom jej shake na 0,2 s. Nie uruchamiaj drżenia co klatkę podczas przytrzymania. Obok pokaż kształt z komponentem FadeOut, który działa niezależnie od skrzynki.

```
// Fragment onUpdate sceny, chest istnieje:  
if (this.input.isKeyPressed(' ')) {  
    GameLab.Tweens.shake(this.chest, 6, 0.2);  
}
```

Sprawdź rozumienie wykresów

Czy 50% czasu zawsze daje 50% drogi? **Nie dla dowolnej krzywej.** W naszych linear i smooth akurat tak.

Czy postęp FadeOut maleje? **Nie: czas rośnie, opacity maleje.**

Ile trwa pełny PingPong przy 0,8 s na odcinek? **1,6 s.**

Jaka jest opacity po 0,5 s przy linearnym FadeOut 2 s? **0,75.**

Czy cancel() oznacza completed = true? **Nie.**

Każdy komponent dodawaj do istniejącego obiektu z właściwym rendererem. Nie łącz dwóch animacji, które równocześnie nadpisują tę samą właściwość. Wykresy w tym kursie wyliczono dokładnie; ilustracja Imagegen pokazuje tylko ideę.